TH-4

4-bit microcontroller with external program memory, 16 × 4-bit RAM and timer

PRELIMINARY · SIMULATION ONLY

Core

- Harvard architecture, 4-bit data, 8-bit instructions
- 17 instructions: arithmetic, logic, memory, branch, wait
- Most instructions execute in 1 cycle, jumps in 2
- Registers: A (accumulator), B (loop counter), Z flag
- 8-bit program counter

Memory

- Up to 256 bytes external parallel program memory
- 16 × 4-bit on-chip RAM

Peripherals

- 4-bit input port
- 4-bit latched output port
- 25-bit timer for delays from 1,024 to 16,777,216 cycles
- Timer runs only during WAIT, for lower power

Implementation

- Written in Verilog for Tiny Tapeout (24 I/O signals)
- 117 flip-flops, 561 cells, about 5,530 µm² in SKY130 (pre-placement estimate)
- Expected to fit in 1 tile (~18,000 µm²)

Tools

- Assembler (Python), Icarus Verilog simulation, self-test of every instruction

Status: TH-4 has been verified in simulation only. It has not been fabricated and has not been through place and route. Maximum clock frequency, electrical characteristics and power consumption have not been measured. These sections will be completed later.

1 Block diagram

On every clock cycle TH-4 drives the program counter (PC) on the ADDR pins, reads the instruction byte from the INSTR pins, decodes it and executes it. The ALU computes the new value of A. RAM, B, the timer and the ports are on chip.

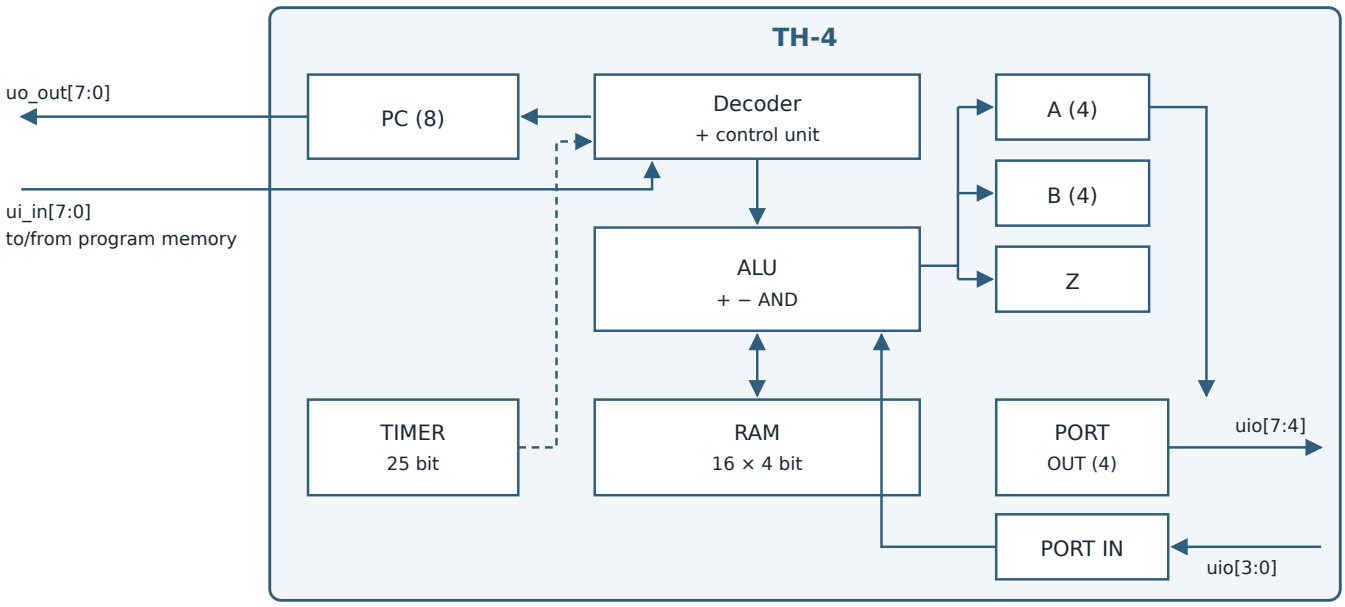


Figure 1-1. TH-4 internal structure. Program memory is external.

2 Pin description

TH-4 uses the Tiny Tapeout interface: 8 inputs, 8 outputs, 8 bidirectional pins, clock and reset.

Tiny Tapeout pin	Name	Direction	Description
uo_out[0]	ADDR0	Output	Program memory address, bit 0 (PC0)
uo_out[1]	ADDR1	Output	Program memory address, bit 1 (PC1)
uo_out[2]	ADDR2	Output	Program memory address, bit 2 (PC2)
uo_out[3]	ADDR3	Output	Program memory address, bit 3 (PC3)
uo_out[4]	ADDR4	Output	Program memory address, bit 4 (PC4)
uo_out[5]	ADDR5	Output	Program memory address, bit 5 (PC5)
uo_out[6]	ADDR6	Output	Program memory address, bit 6 (PC6)
uo_out[7]	ADDR7	Output	Program memory address, bit 7 (PC7)
ui_in[0]	INSTR0	Input	Instruction byte from program memory, bit 0
ui_in[1]	INSTR1	Input	Instruction byte from program memory, bit 1
ui_in[2]	INSTR2	Input	Instruction byte from program memory, bit 2
ui_in[3]	INSTR3	Input	Instruction byte from program memory, bit 3
ui_in[4]	INSTR4	Input	Instruction byte from program memory, bit 4
ui_in[5]	INSTR5	Input	Instruction byte from program memory, bit 5
ui_in[6]	INSTR6	Input	Instruction byte from program memory, bit 6
ui_in[7]	INSTR7	Input	Instruction byte from program memory, bit 7
uio[0]	IN0	Input	Input port, bit 0
uio[1]	IN1	Input	Input port, bit 1
uio[2]	IN2	Input	Input port, bit 2
uio[3]	IN3	Input	Input port, bit 3
uio[4]	OUT0	Output	Output port, bit 0
uio[5]	OUT1	Output	Output port, bit 1
uio[6]	OUT2	Output	Output port, bit 2
uio[7]	OUT3	Output	Output port, bit 3
clk	CLK	Input	Clock. All state changes on the rising edge.
rst_n	RST_N	Input	Reset, active low, synchronous to the clock
ena	–	Input	Not used

uio[7:4] are always outputs and uio[3:0] always inputs (uio_oe = 1111 0000). Direction cannot be changed by software.

3 Memory organization

TH-4 uses a Harvard architecture: program memory and data memory are separate, with separate address spaces.

	Program memory	RAM	Registers
Location	off chip	on chip	on chip
Size	256 × 8 bit	16 × 4 bit	A, B: 4 bit · Z: 1 bit · PC: 8 bit

	Program memory	RAM	Registers
Addresses	0x00 – 0xFF	0x0 – 0xF	not addressable
Contents	instructions	variables	working values
Read by program	no (fetch only)	LOADM, ADDM, SUBM	most instructions
Written by program	no	STORE	most instructions
After reset	unchanged	undefined	all 0

> RAM is not cleared by reset. Software must write a location with STORE before reading it.

The Z flag

Z is set when the result of the last instruction that affects it is 0, and cleared otherwise. It is affected by LOAD, ADD, SUB, AND, IN, LOADM, ADDM, SUBM (based on A) and by DECB (based on B). All other instructions leave it unchanged. There is no carry flag.

4 Instruction set

Instruction format

byte 1								byte 2 (jumps only)
7	6	5	4	3	2	1	0	7 … 0
opcode				n				address aa

The upper 4 bits are the opcode. The lower 4 bits are a number n (0–15): a constant, a RAM address or the WAIT parameter. JUMP, JZ and JNZ have a second byte holding the target address. Opcode F is TAB when n = 0 and WAIT when n = 1–15.

Summary

Hex	Instruction	Operation	Bytes	Cycles	Z
0n	LOAD n	A ← n	1	1	✔
1n	ADD n	A ← A + n	1	1	✔
2n	SUB n	A ← A − n	1	1	✔
3n	AND n	A ← A AND n	1	1	✔
40	IN	A ← IN[3:0]	1	1	✔
50	OUT	OUT[3:0] ← A	1	1	—
60 aa	JUMP aa	PC ← aa	2	2	—
70 aa	JZ aa	if Z = 1: PC ← aa	2	2	—
8n	STORE n	RAM[n] ← A	1	1	—
9n	LOADM n	A ← RAM[n]	1	1	✔
An	LDB n	B ← n	1	1	—
B0	DECB	B ← B − 1	1	1	✔
C0 aa	JNZ aa	if Z = 0: PC ← aa	2	2	—
Dn	ADDM n	A ← A + RAM[n]	1	1	✔
En	SUBM n	A ← A − RAM[n]	1	1	✔
F0	TAB	B ← A	1	1	—
Fn	WAIT n	wait 2^{n+9} cycles	1	2^{n+9} + 2	—

✔ = instruction updates Z. All arithmetic is modulo 16.

Instruction descriptions

Instruction	Description
LOAD n	Loads the constant n into A.
ADD n	Adds n to A. The result is kept to 4 bits (15 + 1 = 0).
SUB n	Subtracts n from A. The result is kept to 4 bits (0 − 1 = 15).
AND n	Bitwise AND. Useful as a mask.
IN	Reads the input port into A.
OUT	Writes A to the output port register. The value is held until the next OUT.
JUMP aa	Unconditional jump to address aa.

Instruction	Description
`JZ aa`	Jumps if the last result was 0.
`STORE n`	Writes A to RAM location n.
`LOADM n`	Loads RAM location n into A.
`LDB n`	Loads n into B.
`DECB`	Decrements B. Z is set when B reaches 0.
`JNZ aa`	Jumps if the last result was not 0. Used with DECB for loops.
`ADDM n`	Adds RAM location n to A.
`SUBM n`	Subtracts RAM location n from A. Z = 1 if they are equal.
`TAB`	Copies A to B.
`WAIT n`	n = 1…15. Clears the timer and halts the program until it has counted 2^{n+9} cycles.

5 Timer and the WAIT instruction

The timer is a 25-bit counter that counts clock cycles. It does not run continuously: it starts from 0 when WAIT n executes, and stops when bit n+9 goes high, that is after 2^{n+9} cycles. Execution then continues with the next instruction. While the timer is not counting, its flip-flops do not toggle.

Every WAIT n takes exactly 2^{n+9} + 2 cycles, independent of what came before. During WAIT, A, B, Z, RAM and the ports do not change.

Instruction	Hex	Cycles	at 1 MHz	at 10 MHz
`WAIT 1`	F1	1,026	1.03 ms	103 µs
`WAIT 2`	F2	2,050	2.05 ms	205 µs
`WAIT 3`	F3	4,098	4.1 ms	410 µs
`WAIT 4`	F4	8,194	8.19 ms	819 µs
`WAIT 5`	F5	16,386	16.4 ms	1.64 ms
`WAIT 6`	F6	32,770	32.8 ms	3.28 ms
`WAIT 7`	F7	65,538	65.5 ms	6.55 ms
`WAIT 8`	F8	131,074	131 ms	13.1 ms
`WAIT 9`	F9	262,146	262 ms	26.2 ms
`WAIT 10`	FA	524,290	524 ms	52.4 ms
`WAIT 11`	FB	1,048,578	1.05 s	105 ms
`WAIT 12`	FC	2,097,154	2.1 s	210 ms
`WAIT 13`	FD	4,194,306	4.19 s	419 ms
`WAIT 14`	FE	8,388,610	8.39 s	839 ms
`WAIT 15`	FF	16,777,218	16.8 s	1.68 s

For longer delays, put WAIT in a loop with B:

```
        LDB 10          ; 10 iterations
wait:   WAIT 15         ; 1.68 s at 10 MHz
        DECB
        JNZ wait        ; about 16.8 s in total
```

6 Input and output ports

Input port (IN[3:0] on uio[3:0])

IN samples the 4 pins when it executes and copies the value to A. There are no internal pull-ups and no debouncing: buttons need external resistors, and bounce must be handled in software (for example with WAIT and a second read).

Output port (OUT[3:0] on uio[7:4])

The output port has its own 4-bit register. OUT writes A to it, and the value stays on the pins until the next OUT, even while A changes. After reset the port is 0000. Pin drive strength is set by the Tiny Tapeout chip; use resistors for LEDs and transistors for larger loads.

7 Reset and clock

Reset (rst_n) is active low and synchronous: it is sampled on the rising clock edge. Hold it low for at least one cycle. While it is low:

- PC = 0x00, A = 0, B = 0, Z = 0
- output port = 0000, timer = 0
- RAM is not changed

When rst_n goes high, execution starts at address 0x00 on the next rising edge.

All registers change on the rising edge of clk. Maximum frequency has not yet been verified. In practice the limit will usually be set by the external memory (section 8).

8 Connecting program memory

Program memory must be asynchronous (combinational): when the address changes, it must output the data without its own clock. TH-4 changes the address on a rising edge and samples the instruction on the next rising edge.

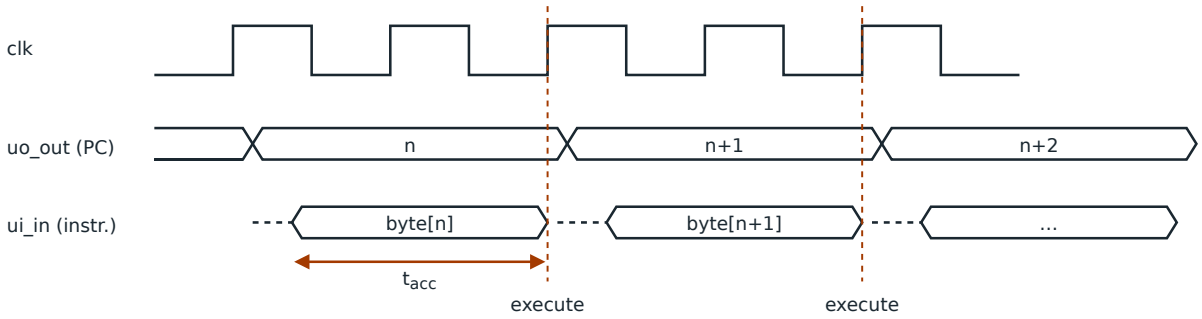


Figure 8-1. Instruction fetch. Memory access time t_{acc} must be less than one clock period, minus pin delays.

Option 1: parallel EEPROM

An EEPROM such as the 28C64 or 28C16: address lines A0–A7 to ADDR0–ADDR7, data lines D0–D7 to INSTR0–INSTR7, remaining address lines to ground, /CE and /OE to ground, /WE to supply. With 150 ns access time the clock must stay below about 6 MHz; leave margin for pin delays (2–4 MHz as a guide). Check that the memory's logic levels match the chip's I/O voltage.

Option 2: microcontroller as memory

An AVR or other microcontroller reads the address on 8 pins, looks up the byte in a table in its flash and drives it on 8 other pins. The TH-4 program is changed simply by reprogramming the microcontroller. Because the lookup is done in software, the TH-4 clock must be slow enough for the microcontroller to keep up (to be measured for each implementation).

9 Development tools

File	Purpose
`tt_um_th4.v`	TH-4 in Verilog
`th4asm.py`	assembler: .asm → .hex
`run.v`	simulates any program from prog.hex, with an execution trace
`tb.v, selftest.asm`	self-test: exercises every instruction and the timer

Assembler syntax

```
; comment
temp = 0            ; name for RAM location 0 (or .equ temp = 0)
loop:   OUT         ; label: the instruction's address
        STORE temp
        JUMP loop
        ORG 0x40    ; following code starts at address 0x40
```

Workflow

```
python3 th4asm.py program.asm > prog.hex
iverilog -g2012 -o run run.v tt_um_th4.v
vvp run +cycles=500 +in=5      ; +in = input port value
```

10 Examples

All examples have been verified in simulation.

Counter with timer

Counts 0–15 on the output port, one step every 0.42 s at 10 MHz.

```
        LOAD 0
loop:   OUT
        WAIT 13
        ADD 1
        JUMP loop
```

Machine code: `00 50 FD 11 60 01`

Running light

One LED at a time: 0001, 0010, 0100, 1000. A + A shifts left, and the overflow (8 + 8 = 0) restarts the cycle.

```
temp = 0
        LOAD 1
loop:   OUT
        WAIT 13
        STORE temp
        ADDM temp       ; A = A + A
        JNZ loop
        LOAD 1
        JUMP loop
```

Machine code: `01 50 FD 80 D0 C0 01 01 60 01`

Button

If IN0 is 1 the output shows 1111, otherwise 0000.

```
loop:   IN
        AND 1           ; keep bit 0 only
        JZ off
        LOAD 15
        OUT
        JUMP loop
off:    LOAD 0
        OUT
        JUMP loop
```

Machine code: `40 31 70 08 0F 50 60 00 00 50 60 00`

11 Implementation data

Item	Value
Flip-flops	117 (64 RAM, 53 registers and control)
SKY130 cells (sky130_fd_sc_hd)	561
Cell area	about 5,530 µm²
Tiny Tapeout tiles	1 (expected, not yet verified)

Flip-flops per unit

Unit	Flip-flops	Contents
`RAM`	64	16 locations × 4 bit
`Timer`	25	WAIT instruction counter
`PC`	8	instruction address
`A`	4	accumulator
`B`	4	loop counter
`port_out`	4	output port register
`jop`	4	pending jump type
`state`	3	control state (one-hot: FETCH, ADDR, WAIT)
`Z`	1	zero flag
Total	**117**	

RAM is 55% of the flip-flops and the timer 21%. The core (PC, A, B, Z, port, control) is 28. state is declared as 2 bits in the source, but synthesis encodes it one-hot with 3 flip-flops.

Measured with Yosys 0.33 and the sky130_fd_sc_hd library (tt, 25 °C, 1.8 V), before placement. Final area will be larger due to routing and clock buffers.

12 Revision history

Revision	Changes
1.0	16 instructions, A and B registers, 16 × 4-bit RAM.
1.1	Timer and WAIT n instruction (opcode F, n = 1–15).
1.2	The timer runs only during WAIT, to save power. Every WAIT now takes exactly 2^{n+9} + 2 cycles.

Design: George Avrsam, 2026.